Universität Konstanz

Introduction to Computing for the Social Sciences

Session 9 – Theoretical Computer Science

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning (8.01)
- Session 11: Databases
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Outline

1. Finite-state Automata
2. Formal Languages and Grammars
3. Turing Machines
4. Computability
5. Complexity Classes

The regular expression that took down a large part of the web

Universität Konstanz

MOTIVATING EXAMPLE

On 2 July 2019 Cloudflare deployed **one line**: a pattern meant to spot malicious JavaScript. On some inputs it made the matcher try an **exponentially growing** number of possibilities. CPU across the global network went to 100%, and a large share of the web was unreachable for **27 minutes**.

1 line
of pattern deployed

100%
CPU, worldwide, within seconds

27 min
global outage

A pattern matcher is a machine with a precise definition. What can it do, what can it not, and what makes it explode?

Cloudflare, “Details of the Cloudflare outage on July 2, 2019”

300 students, 40 seminars, and no schedule

Universität Konstanz

MOTIVATING EXAMPLE

Every student ranks the seminars; every seminar has a cap; some combinations clash. You want an assignment that satisfies the constraints. The department has done this by hand for years. Written down properly, the number of possible assignments exceeds the number of atoms in the observable universe.

- Trying harder does not help. Buying a faster computer does not help.
- There is a **mathematical reason**, and it is one of the seven Millennium Prize Problems.

How can we *prove* a problem is hard, rather than concluding that we were not clever enough?

Can a program check your program?

Universität Konstanz

MOTIVATING EXAMPLE

A marking system receives 300 student submissions and would like to reject the ones that loop forever, before running them. This sounds like a matter of engineering effort. In **1936** Turing proved that **no program can do this**, for any programming language, ever.

- Not “too slow”. Not “not yet”. **Impossible**, and provably so.
- The proof is one page, and assumes only that the checker exists.

What are the limits of computation itself, and how would you prove something *cannot* be done?

Turing, “On Computable Numbers, with an Application to the Entscheidungsproblem”, *Proc. London Math. Soc.* (1936)

Motivation

Universität Konstanz

Theoretical computer science questions:

- what can we compute?
- what are the principles of computing machines?
- how does a computing machine have to be structured to perform (any) computing task?

Topics of theoretical computer science:

- formal definitions of languages and their classification
- ways of formalizing computation as an abstract process of transition between states of a computer program independent of its implementation
- proofs of computability based on these theoretical constructs

Modeling computation through automata

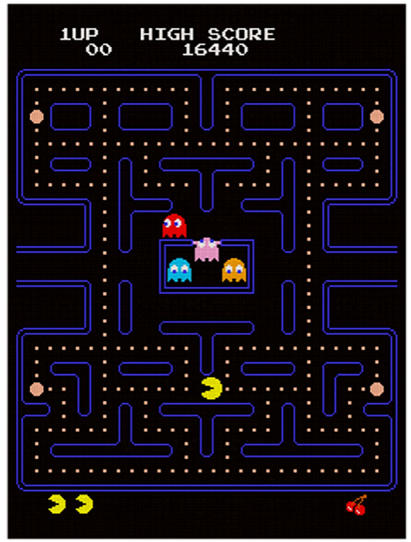
Universität Konstanz

Fundamental understanding of the theory of computing is important:
How do we model computing? How do we abstract this process?

- modeling **input**: breaks down to accepting a finite sequence of bits
- modeling **output**: from a finite sequence of bits we can, in principle, constitute any output (numbers, graphics, sound etc.)
- modeling **processing**:
  - in principle, bit-by-bit operations but we need memory to keep track of intermediate steps
  - “power” of computing models ultimately changes with their ability to memorize intermediate steps
  - and we need rules that define how we transition from one operation to the next, e.g. calculating machines are based on arithmetic operations
  - more generally, a computing machine must have rules to transition between a finite number of internal states

Pac-Man ghosts as automata

- Pac-Man ghosts have actions depending on their **state**:
  - Wander around the maze
  - Chase Pac-Man
  - Flee from Pac-Man
  - Go to base and resurrect
- Their actions depend on the **input** produced by Pac-Man and the game, for example:
  - Pac-Man gets close to a ghost
  - Pac-Man goes far from a ghost
  - Pac-Man eats a special pellet
  - Pac-Man touches ghost (losing)
  - Pac-Man eats ghost (with special pellet)
  - special pellet expires
  - Pac-Man eats all pellets



Ghosts are **automata**: they have states and transitions between states that depend on the input

Finite-state automaton

Universität Konstanz

Definition: deterministic finite state automaton (DFA)

A deterministic finite state automaton is defined by a 5-tuple $M = (Q, \Sigma, \delta, q_0, X)$ consisting of:

- Q, a finite set of **states**
- Σ, a finite set of input symbols called the **alphabet**, $Q \cap \Sigma = \emptyset$
- $\delta : Q \times \Sigma \to Q$, a **transition function**
- $q_0 \in Q$, an **initial** or start state
- $X \subseteq Q$, a set of **accept states**

Finite-state automaton

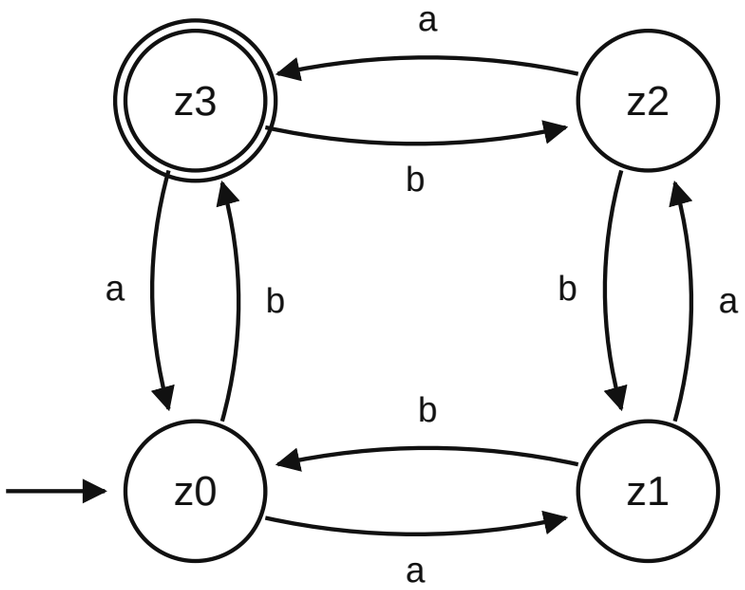
Universität Konstanz

Functioning of automaton:

- is initially in start state q_0
- input word is read character-by-character (or command-by-command)
- automaton transitions from state to state according to each character input
- if automaton ends up in an accept state, the word (or command sequence) is part of the language
- if not, it is not part of the language
- Convention: if a transition is not defined for a state and character input, then the automaton goes to a non-terminal reject state

Finite-state automaton: example

- $Q = \{z_0, z_1, z_2, z_3\}$
- $\Sigma = \{a, b\}$
- $q_0 = z_0$
- $X = \{z_3\}$
- δ:
  - $\delta(z_0, a) = z_1$
  - $\delta(z_0, b) = z_3$
  - $\delta(z_1, a) = z_2$
  - $\delta(z_1, b) = z_0$
  - $\delta(z_2, a) = z_3$
  - $\delta(z_2, b) = z_1$
  - $\delta(z_3, a) = z_0$
  - $\delta(z_3, b) = z_2$



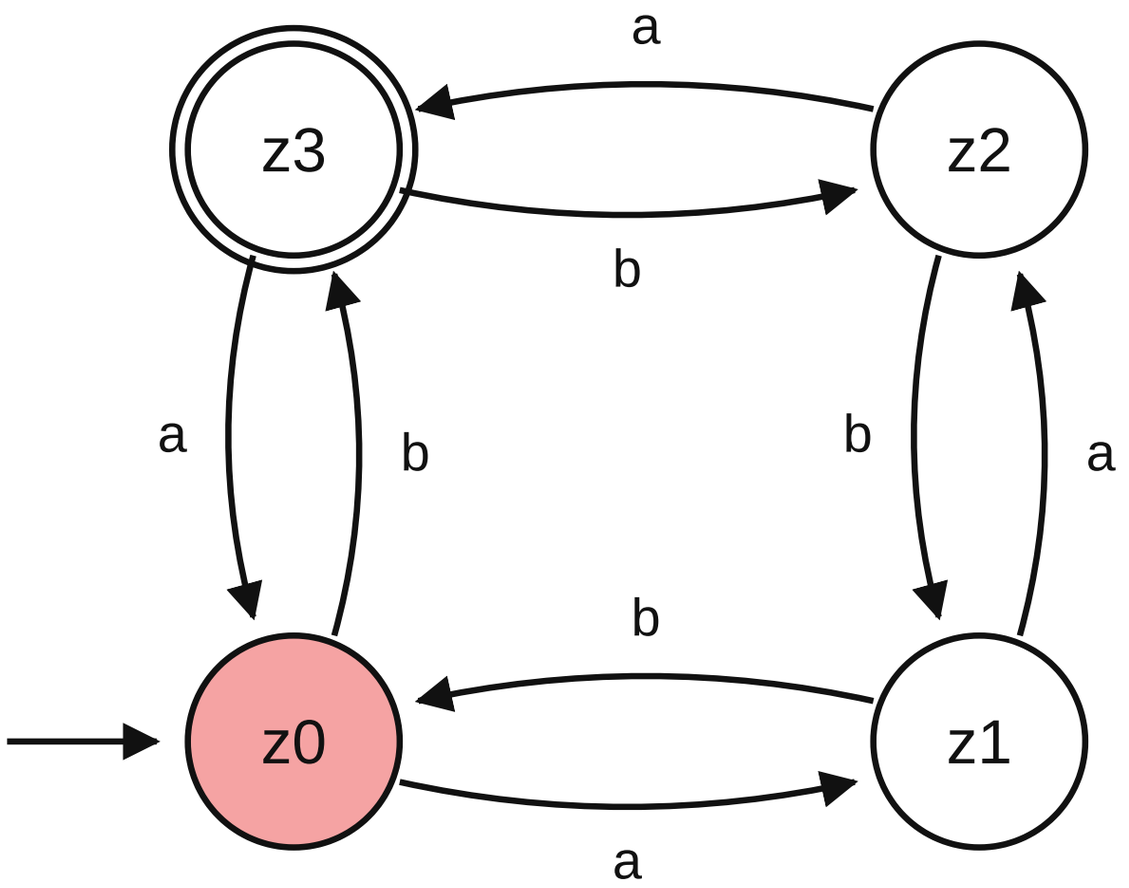
Examples:

`aaa`	finished
`aaaab`	finished
`bbaaabaaa`	finished
`aa`	not finished
`abba`	not finished
`bbb`	not finished

(number of *a*'s in *y*) –
(number of *b*'s in *y*)
$\equiv 3 \pmod 4$

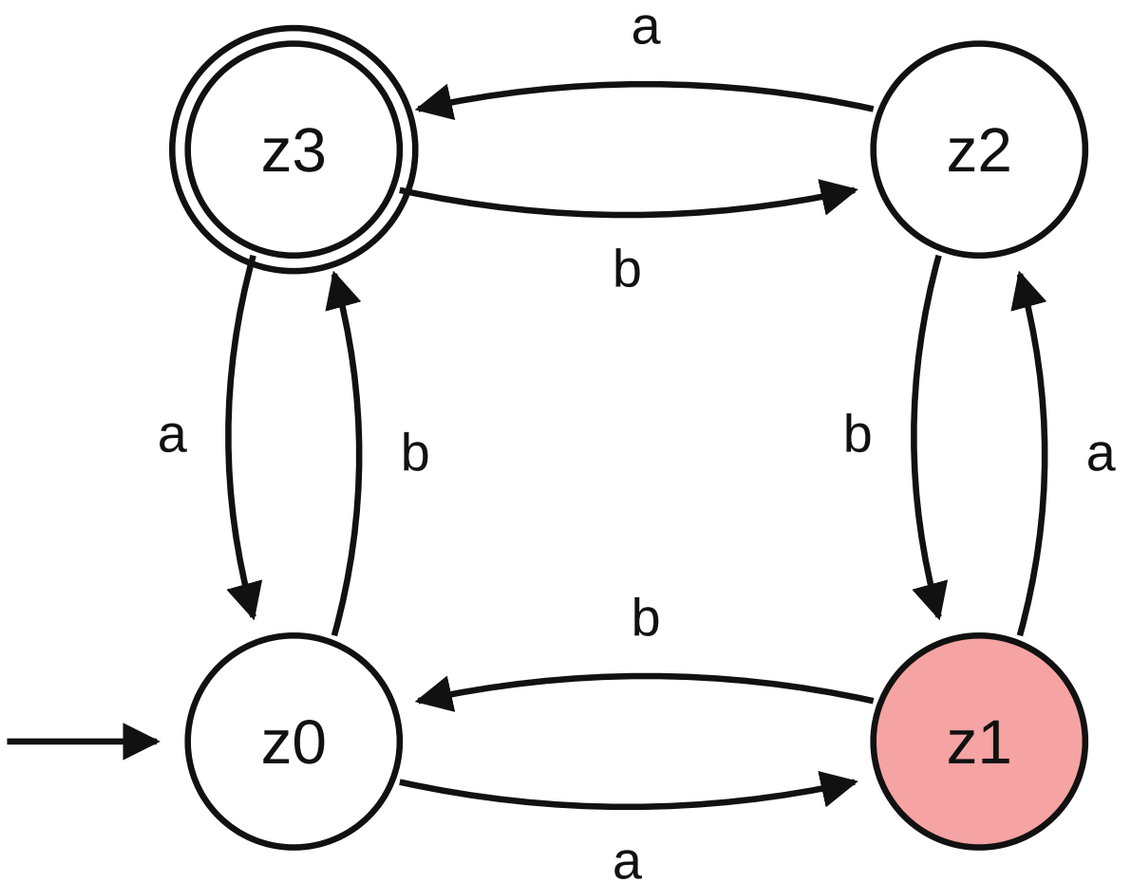
Finite-state automaton: example

Input: abb



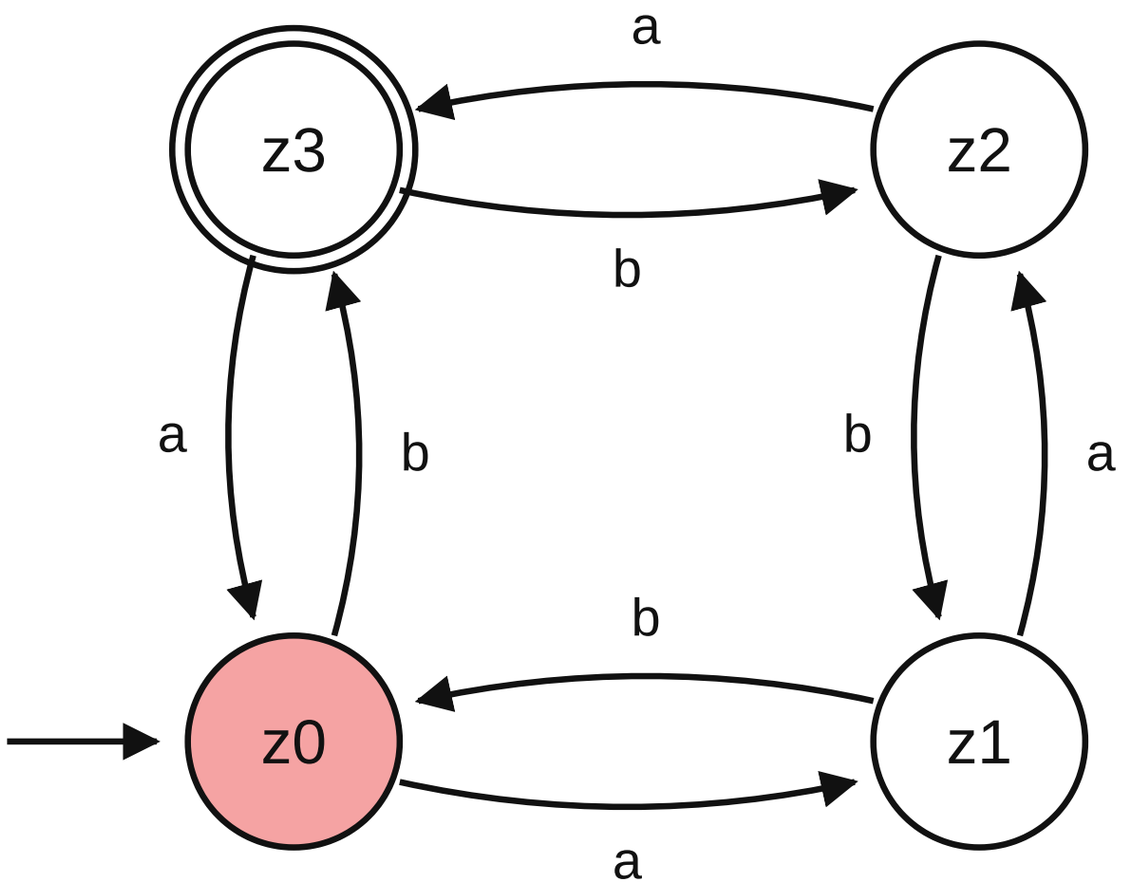
Finite-state automaton: example

Input: abb



Finite-state automaton: example

Input: abb

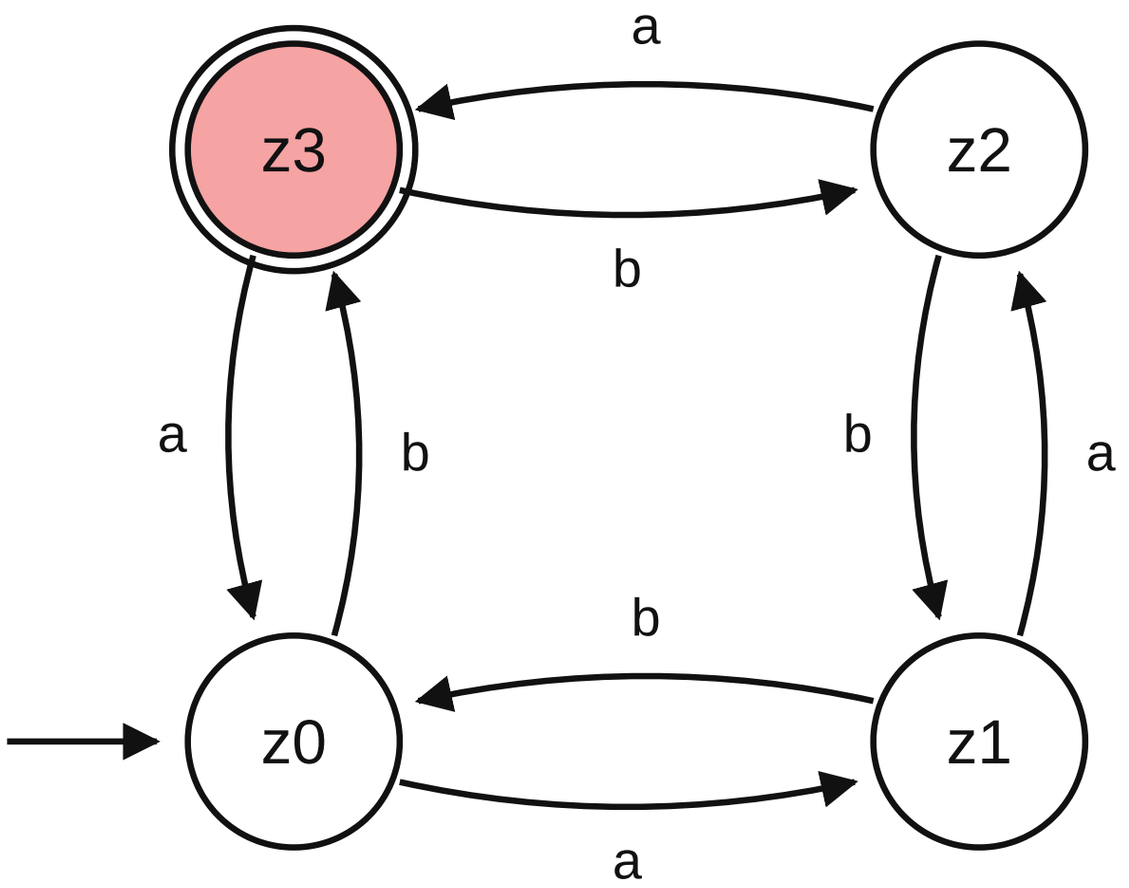


Finite-state automaton: example

Input: abb

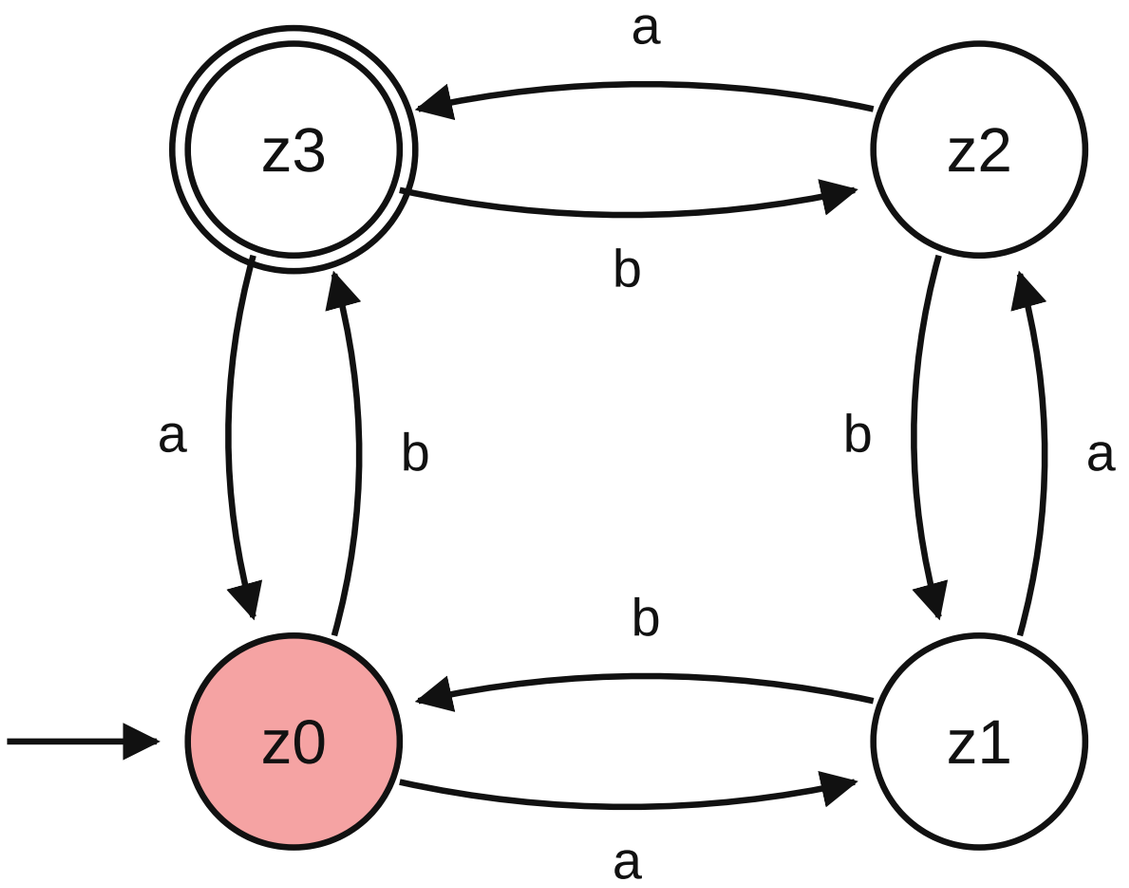
z3 is an accept state: input is recognized

(abb is part of the language recognized by the DFA)



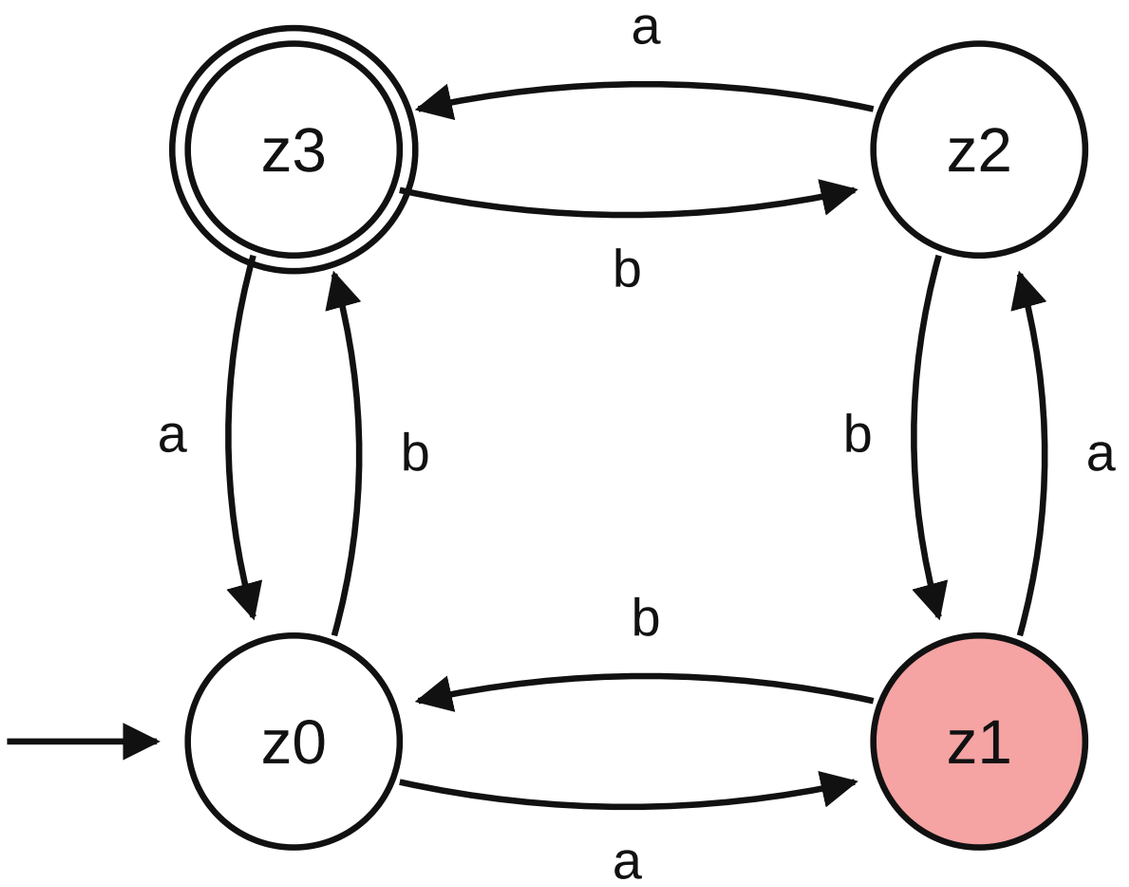
Finite-state automaton: example 2

Input: aab



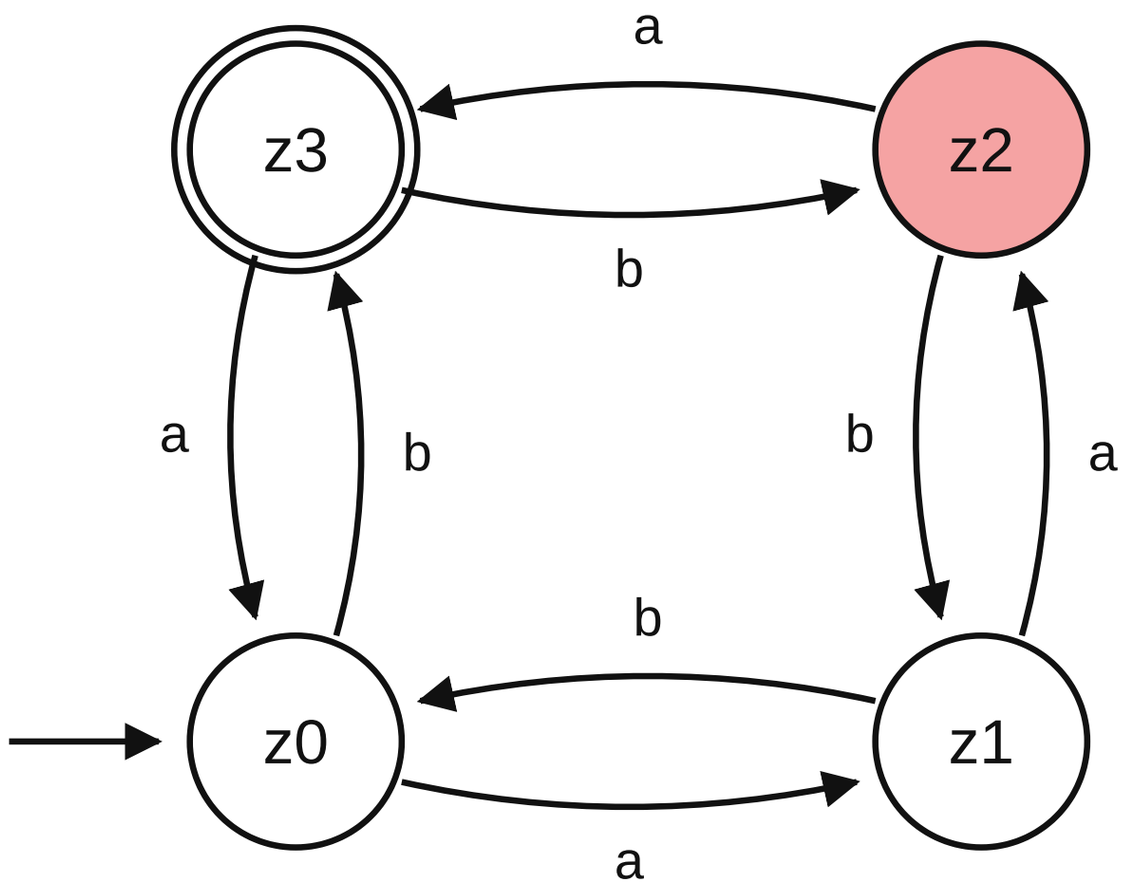
Finite-state automaton: example 2

Input: aab



Finite-state automaton: example 2

Input: aab

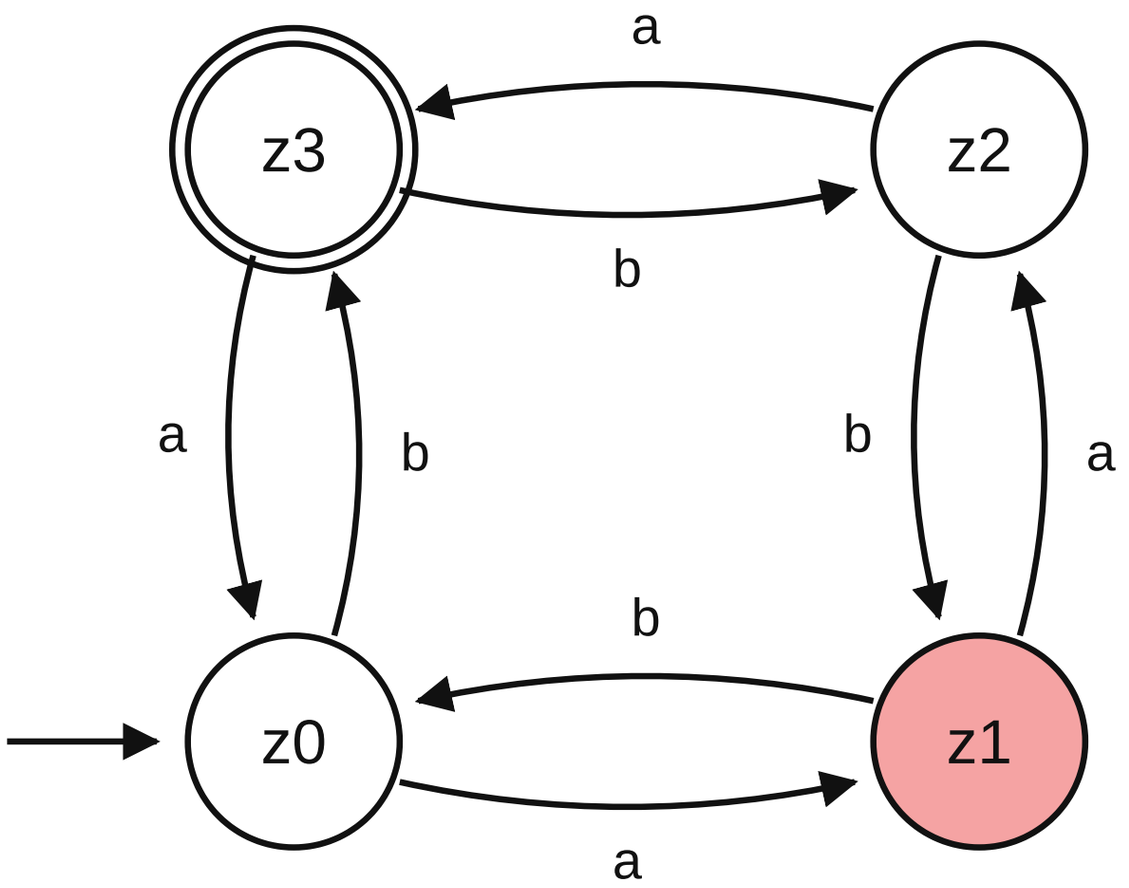


Finite-state automaton: example 2

Input: `aab`

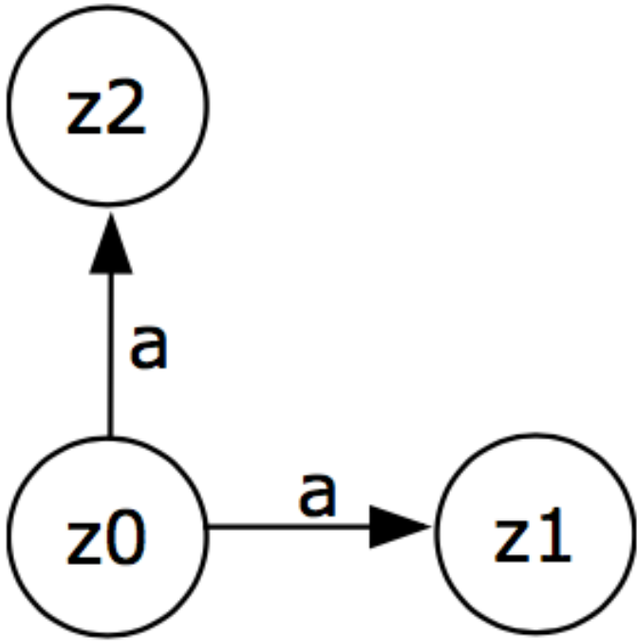
z1 is not an accept state: input is not recognized

(`aab` is not part of the language recognized by the DFA)



Nondeterministic finite-state automata

- So far we have considered **deterministic** state changes:
  - every input character (or command) has maximally one state change
- A **nondeterministic** automaton can “select” among transitions
- accepts any word that has **at least one path** from the start state q_0 to an accept state X



Nondeterministic finite-state automaton

Universität Konstanz

Definition: non-deterministic finite state automaton (NFA)

A non-deterministic finite state automaton is defined by a 5-tuple $M = (Q, \Sigma, \delta, q_0, X)$ consisting of:

- Q, a finite set of states
- Σ, a finite set of input symbols called the alphabet, $Q \cap \Sigma = \emptyset$
- $\delta : Q \times \Sigma \to \mathcal{P}(Q)$, a transition function, where $\mathcal{P}(Q)$ is the power set of Q (the set of all subsets of Q)
- $q_0 \in Q$, an initial or start state
- $X \subseteq Q$, a set of accept states

An NFA can also have ε-transitions, taken without reading any input symbol

Theorem (Rabin, Scott): any NFA can be simulated by a DFA

- in other words, NFAs and DFAs are equivalent
- a DFA might have many more states; it is easier to write and reason with NFAs

Formal Languages and Grammars

Universität Konstanz

1. Finite-state Automata
2. **Formal Languages and Grammars**
3. Turing Machines
4. Computability
5. Complexity Classes

Grammar

Universität Konstanz

- Initial motivation: How do we ensure that a command (or input) is a syntactically correct part of a given language?
- Gave rise to the field of **automata theory** in theoretical computer science
  - informally: an automaton is a state space representation of a program, i.e., the description of program flow through incremental transitions between some (elementary) program states
- Grammars help us to answer the question:
  - **What can be computed with a finite-state automaton?**

A grammar is a formal set of rules from which you can derive all inputs that the automaton accepts and no input that it does not accept

Nomenclature about formal languages

Universität Konstanz

- **Kleene star operator $*$**
  - operation on a given set V
  - amounts to the concatenation of V on itself, i.e., any finite combination of the elements of V
  - example: $V = \{+, 1, 2\}$ then
    $V^* = \{\varepsilon, +, 1, 2, +1, +2, 2+, 12, 1+2, +12, ++++, \text{etc.}\}$
  - V^+ is V^* without the empty word ε
- **Formal Language L**
  - set of strings of symbols formed from its alphabet Σ
  - formally: $L \subseteq \Sigma^*$
  - abstract definition: in practice, a well-formed (programming) language further requires the definition of an appropriate grammar G that limits which words are syntactically permitted

Grammar

A finite set of formal rules for generating syntactically correct sentences.

Definition: Grammar

A Grammar is a 4-tuple $G = (N, \Sigma, P, S)$ that satisfies the following conditions:

- N is a finite set of all **variables** (or nonterminal symbols), symbols which take part in the generation of the sentence but are not a component of the sentence (denoted by capital letters)
- Σ is a finite set: the alphabet of **terminal symbols** such that $N \cap \Sigma = \emptyset$. Symbols which are the components of the sentences generated using a grammar (denoted by small letters)
- P is the finite set of **production rules**, $P \subseteq (N \cup \Sigma)^+ \times (N \cup \Sigma)^*$, that map one string of symbols to another (denoted by "$\rightarrow$")
- $S \in N$ is the **start variable**

Grammar: example

- Grammar that generates proper bracketed arithmetic expressions:
  - $N = \{Expr, Term, Fact\}$
  - $\Sigma = \{(,), a, +, *\}$
  - $S = Expr$

$P = \{$

1. $Expr \rightarrow Term$
2. $Expr \rightarrow Expr + Term$
3. $Term \rightarrow Fact$
4. $Term \rightarrow Term * Fact$
5. $Fact \rightarrow a$
6. $Fact \rightarrow (Expr)$ $\}$

- This grammar produces correct arithmetic expressions with parentheses
  - for example: $a * (a + a) + a$

Grammar: derivation example

- $N = \{Expr, Term, Fact\}$,
  $\Sigma = \{(,), a, +, *\}$, $S = Expr$

$P = \{$

1. $Expr \rightarrow Term$
2. $Expr \rightarrow Expr + Term$
3. $Term \rightarrow Fact$
4. $Term \rightarrow Term * Fact$
5. $Fact \rightarrow a$
6. $Fact \rightarrow (Expr)$ $\}$

Derivation of $a * (a + a) + a$

rule	
	$S = Expr$
(2)	$Expr + Term$
(1)	$Term + Term$
(4)(3)	$Term * Fact + Fact$
(3)(6)(5)	$Fact * (Expr) + a$
(5)(2)	$a * (Expr + Term) + a$
(1)(3)	$a * (Term + Fact) + a$
(3)(5)	$a * (Fact + a) + a$
(5)	$a * (a + a) + a$

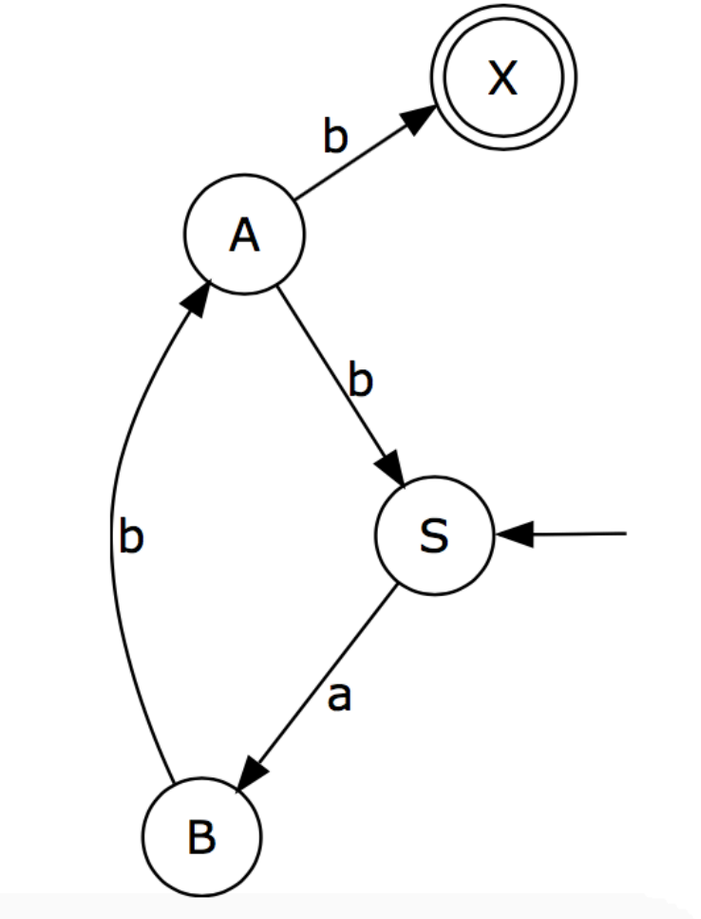
Grammar $\equiv$ Automaton

A grammar $G = (N, \Sigma, P, S)$ whose rules have the form $A \to aB$ or $A \to a$ (a *regular* grammar) is equivalent to an automaton $M = (Q, \Sigma, \delta, q_0, \{X\})$ with the following mapping:

- the automaton uses the same input alphabet Σ
- $Q = N \cup \{X\}$, where $X \notin N$, i.e., the states of the automaton correspond to the variables of the grammar plus an additional state X
- $q_0 = S$, i.e., the initial state derives from the start variable
- the set of accept states is $\{X\}$, i.e., the only accept state is the new state X
- $\delta(A, a) \ni B$ if $A \to aB \in P$, i.e., there exists a transition from state A to state B if the input is a
- $\delta(A, a) \ni X$ if $A \to a \in P$, i.e., there exists a transition from state A to the final state X if the input is a

Grammar $\equiv$ Automaton: example

- Example transformation for grammar G with:
  - $N = \{S, A, B\}$
  - $\Sigma = \{a, b\}$
  - $P = \{\, S \to aB,$
    $B \to bA,$
    $A \to b,$
    $A \to bS \,\}$



Formal Languages

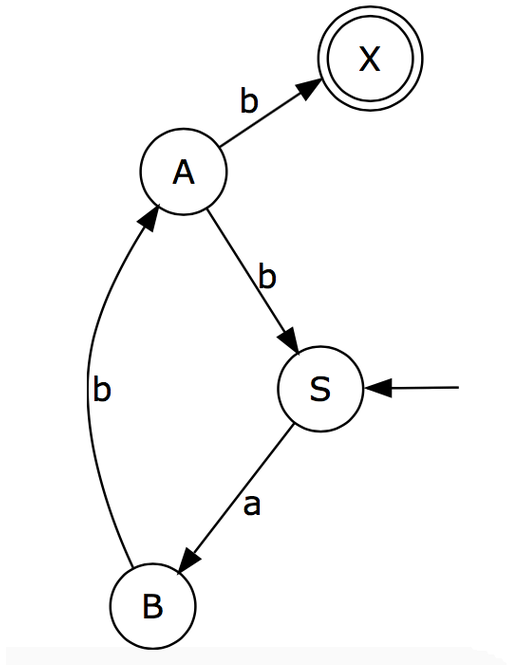
Universität Konstanz

Definition: Formal Language

The language L defined by G is $L(G) = \{ w \in \Sigma^* \mid S \Rightarrow_G^* w \}$, i.e., G derives w in zero or more steps from the start symbol S.

For example, the grammar with

- $N = \{S, A, B\}$
- $\Sigma = \{a, b\}$
- $P = \{S \to aB,\ B \to bA,\ A \to b,\ A \to bS\}$



defines the language:

$$L(G) = \{(abb)^n \mid n \geq 1\}$$

Regular Languages

Universität Konstanz

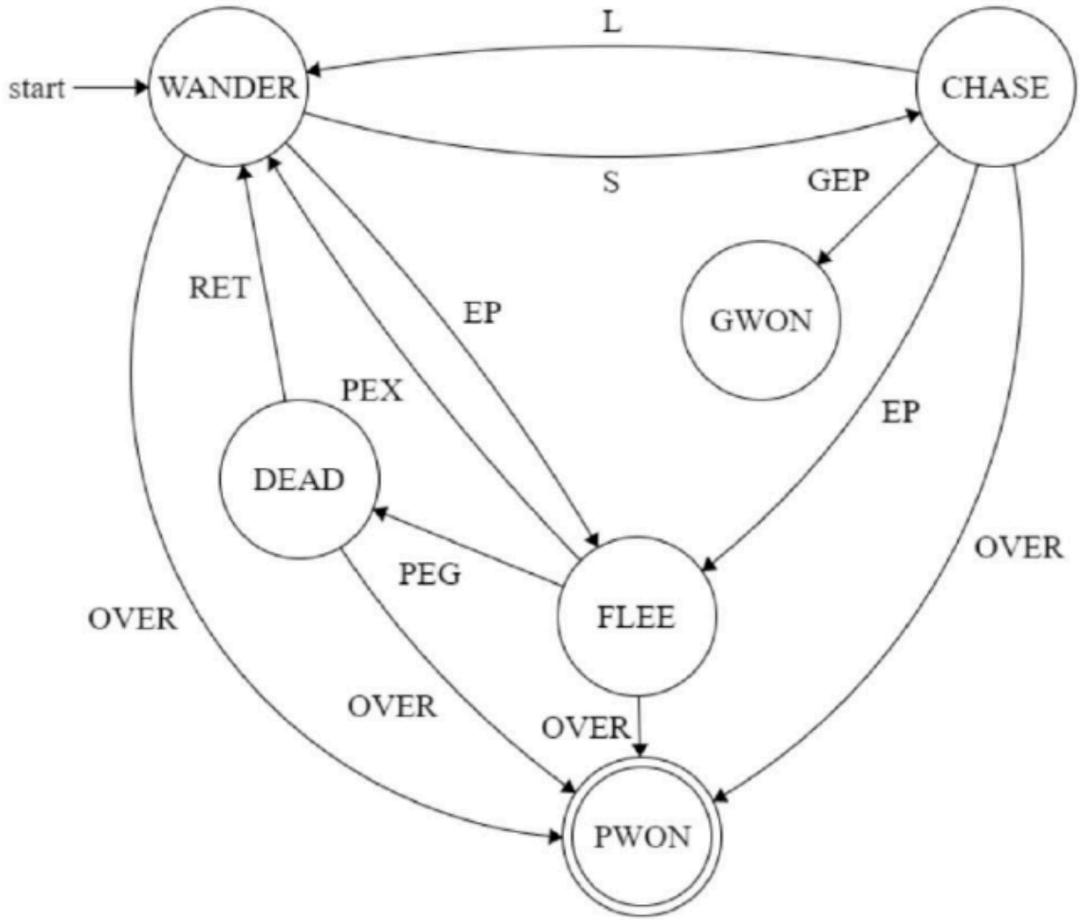
Theorem

Every language recognizable by finite automata is a **regular language**.

Regular Expressions (RE): R is a regular expression if R is:

1. a, such that $a \in \Sigma$ (just a single character in the alphabet)
2. ε (the empty word)
3. $\emptyset$ (the empty set)
4. $R_1 \mid R_2$, where R_1 and R_2 are regular expressions (alternative RE)
5. $R_1 R_2$, where R_1 and R_2 are regular expressions (concatenation of RE)
6. R_1^*, where R_1 is a regular expression (repetition of RE)

Pac-Man ghosts DFA



Emulating Pac-Man using Finite State Machines. Soumyabrata Bairagi, Bachelor Thesis, Indian Institute of Information Technology, Ranchi.

Outtake – Social Science Applications

Universität Konstanz

For each revision of each page in the Wikipedia dump, we used the following regular expression in Python[8] to extract *wilinks*:

```
\[\[
(?P<link>
    [^\n\|\]\[\<\>\{\}]{0,256}
)
(?:
  \|
  (?P<anchor>
      [^\[]*?
  )
)?
\]\]
```

[8] https://github.com/WikiLinkGraphs/wikidump/blob/70b0c7f929fa9d66a220caf11c9e31691543d73f/wikidump/extractors/misc.py#L203

WikiLinkGraphs: A Complete, Longitudinal and Multi-Language Dataset of the Wikipedia Link Networks. Cristian Consonni, David Laniado, Alberto Montresor. *Proceedings of the International Conference on the Web and Social Media* (2019)

Outtake – Social Science Applications



XKCD: Regular Expressions https://xkcd.com/208/

Outtake – Social Science Applications



XKCD: Regular Expressions https://xkcd.com/208/

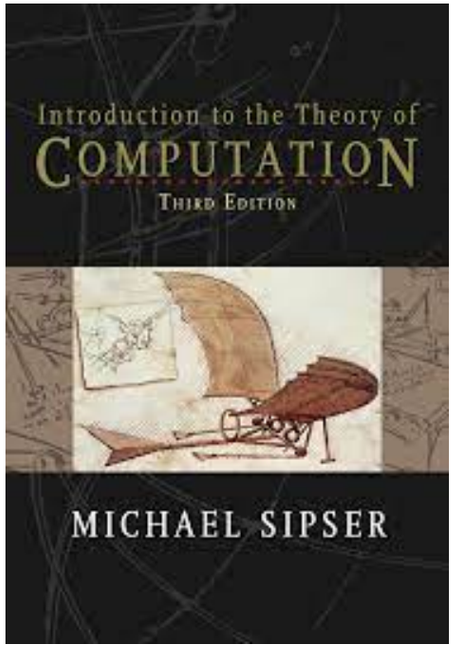
Outtake – Social Science Applications



XKCD: Regular Expressions https://xkcd.com/208/

Further Readings

Chapters 1 & 2 of
Introduction to the Theory of Computation,
by Michael Sipser



Chapters contain additional proofs and lemmas, but also everything else seen here

Turing Machines

Universität Konstanz

1. Finite-state Automata
2. Formal Languages and Grammars
3. **Turing Machines**
4. Computability
5. Complexity Classes

The problem with DFA and the Chomsky Hierarchy

Universität Konstanz

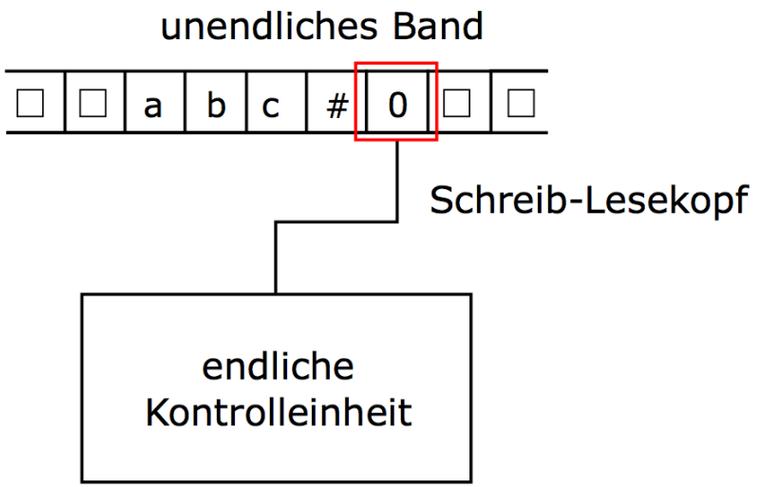
A simple language that a DFA cannot recognize:

$$L = \{a^n b^n \mid n \geq 1\}: \quad S \to ab \mid aSb$$

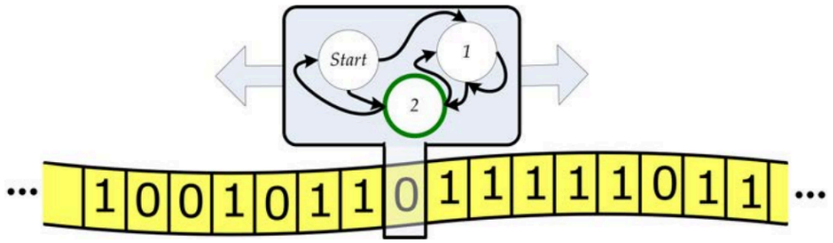
Type	Type	Automaton	Rules	Rule	Recognizes
0	Recursively enumerable	Turing machine, infinite memory	α, β arbitrary; left at least one non-terminal	$\alpha \to \beta$	all
1	Context sensitive language	Linear bounded Turing machine	left at least one non-terminal; left shorter	$\alpha A \beta \to \alpha \gamma \beta$	$a^n b^n c^n$
2	Context free language	Pushdown automaton, stack	left only one non-terminal	$A \to \alpha$ (α string of terminals and non-terminals)	$a^n b^n$
3	Regular language	Finite state automaton	left only one non-terminal	$A \to aB$ or $A \to Ba$	a^n

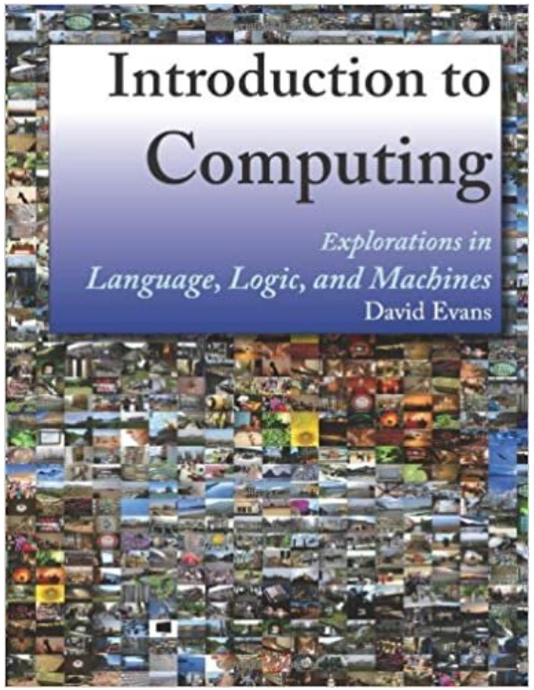
Turing Machines

- Turing Machines (TM) have a **tape** consisting of infinite cells on which each cell either contains an input symbol or a special symbol called *blank*
- The machine can read and write those symbols, one at a time
- The machine has a **head** pointer which points to the cell currently being read and it can move in both directions
- The tape head can move left and right
- The **state register** stores the current state of the machine
- Set of states is finite
- Table of (finite) instructions describing how the machine reads, writes and changes state



Turing Machines: Recommended Readings





- Illustration of Turing Machine from *Introduction to Computing* by Evans (2011)
- Recommended readings:
  - Chapter 6 for Turing Machines
  - Chapter 12 for Computability
- Sipser (2012) Ch. 7 for Complexity Classes

Turing Machines: Formalization

Universität Konstanz

Definition: Turing machine

A Turing machine is defined by a 7-tuple $TM = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ consisting of

- Q: a finite set of **states**
- Σ: a finite set of symbols called the **input alphabet**, $Q \cap \Sigma = \emptyset$ (symbols which are part of the input alphabet)
- Γ: a finite set of symbols called the **working alphabet**, $\Sigma \subset \Gamma$ (symbols which can be written on the tape)
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, N\}$, the **transition function** in the deterministic case;
  $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R, N\})$ in the nondeterministic case
- $q_0 \in Q$: an initial or **start state**
- $\square \in \Gamma$: the **blank symbol** (initially, tape is filled with $\square$ except the input)
- $F \subseteq Q$: the set of **final states** (if any state of F is reached, input is accepted)

Turing Machines: Formalization

Universität Konstanz

Definition: Turing machine

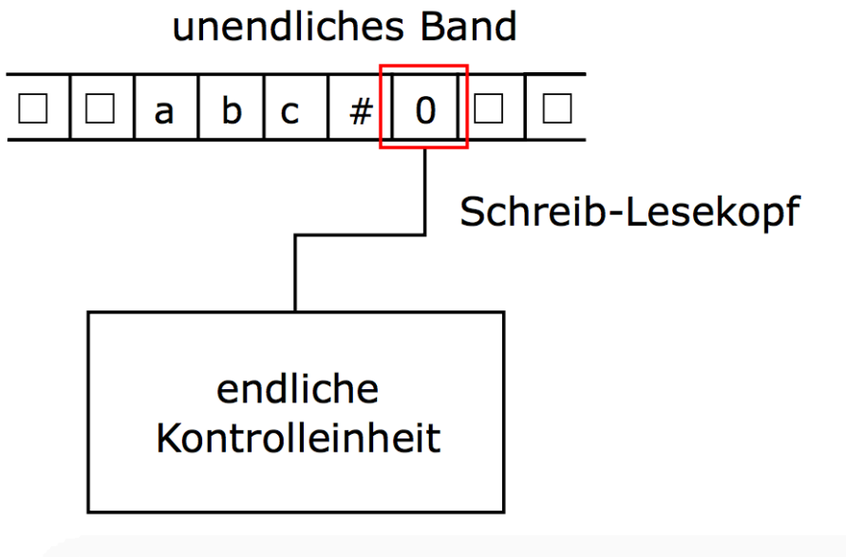
A Turing machine is defined by a 7-tuple $TM = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ consisting of

- Q: a finite set of **states**
- Σ: a finite set of symbols called the **input alphabet**, $Q \cap \Sigma = \emptyset$ (symbols which are part of the input alphabet)
- Γ: a finite set of symbols called the **working alphabet**, $\Sigma \subset \Gamma$ (symbols which can be written on the tape)
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, N\}$, the **transition function** in the deterministic case;
  $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R, N\})$ in the nondeterministic case
- $q_0 \in Q$: an initial or **start state**
- $\square \in \Gamma$: the **blank symbol** (initially, tape is filled with $\square$ except the input)
- $F \subseteq Q$: the set of **final states** (if any state of F is reached, input is accepted)

From (state, tape symbol read) to (next state, tape symbol written, movement of head). Movements are Left, Right, None. It can have special operations like Halt and Error.

Functioning of Turing Machines

- Transition example: $\delta(z, a) = (z', b, x)$
  - TM is in state z, and
  - a is underneath the tape head
  - TM transitions to state z', writes b in the cell under the head
  - and after that moves the head according to $x \in \{R, L, N\}$
- **initial state**
  - only the input is written on the tape
  - tape head on first input symbol/element
  - all unused places are marked with □. # is an alternative notation for this



Turing Machine example: checking parentheses

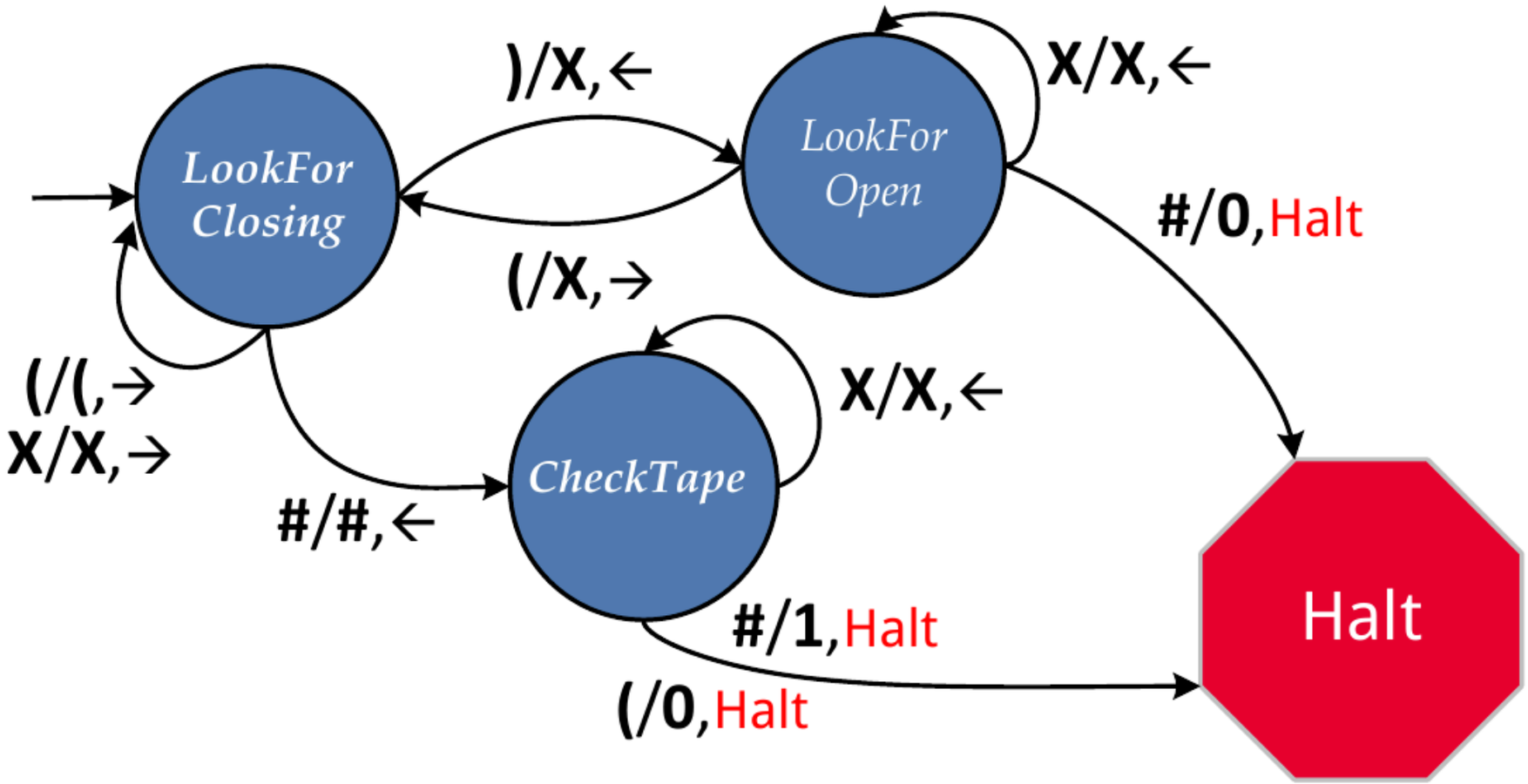
- **Problem:** Check that parentheses in the input are balanced
  - e.g. `()(())(()())` is balanced but `(((()(` is not
- **Input:** tape contains the string of parentheses with `#` at the beginning and end to mark limits
- **Output:** there is a `1` on the tape if the input is balanced, or a `0` otherwise
  - Only what is written on the cell of the head matters
- **Strategy:**
  - look for a closing parenthesis and replace with `X`
    - then find its matching opening parenthesis and replace with `X`
    - if you find `#` instead of a matching opening parenthesis: write `0`
  - Repeat until you reach `#` when looking for a closing parenthesis
  - If tape is all `X` between `#`, write `1`

Turing Machine example: checking parentheses

State	Read	Next State	Write	Move	
LookForClosing	)	*LookForOpen*	X	←	*Found closing.*
LookForClosing	(	*LookForClosing*	(	→	*Keep looking.*
LookForClosing	X	*LookForClosing*	X	→	*Keep looking.*
LookForClosing	#	*CheckTape*	#	←	*End of tape.*
LookForOpen	)	-	X	Error	*Shouldn't happen.*
LookForOpen	(	*LookForClosing*	X	→	*Found open.*
LookForOpen	X	*LookForOpen*	X	←	*Keep looking.*
LookForOpen	#	-	0	Halt	*Reached beginning.*
CheckTape	)	-	0	Error	*Shouldn't happen.*
CheckTape	(	-	0	Halt	*Unbalanced open.*
CheckTape	X	*CheckTape*	X	←	*Keep checking.*
CheckTape	#	-	1	Halt	*Finished checking.*

From *Introduction to Computing* by Evans (2011)

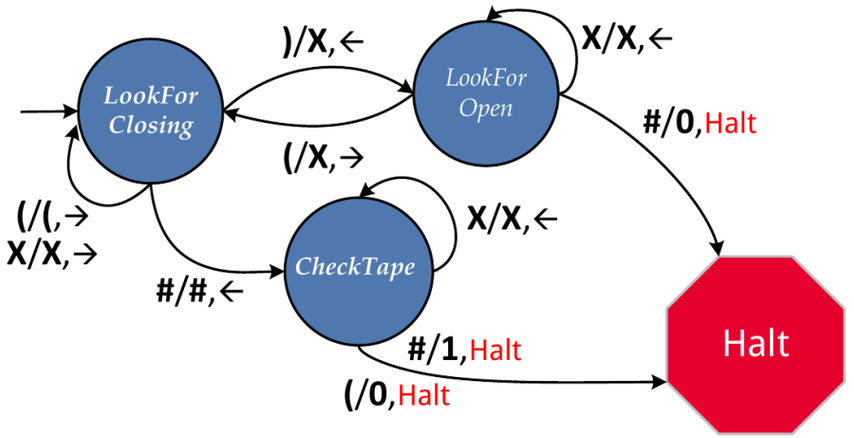
Turing Machine example: checking parentheses



From *Introduction to Computing* by Evans (2011)

Turing Machine tape trace example

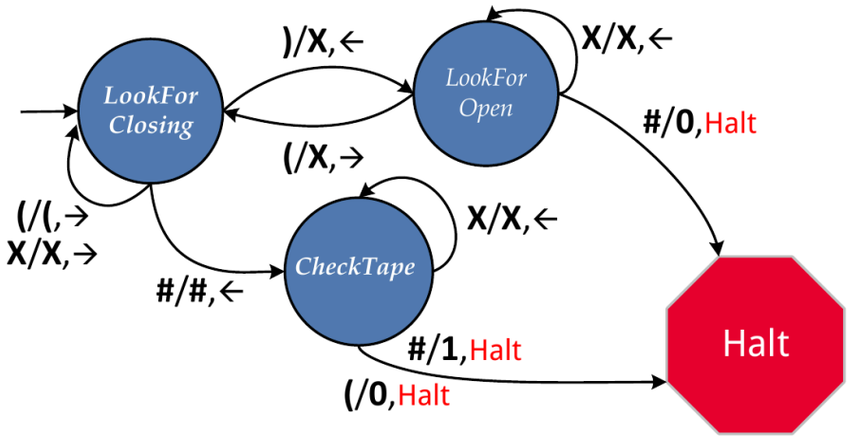
#	(	(	)	(	)	)	#



- Input string to process
- TM starts in leftmost cell of input
- TM starts in state `LookForClosing`
- TM moves without rewriting until first `)`

Turing Machine tape trace example

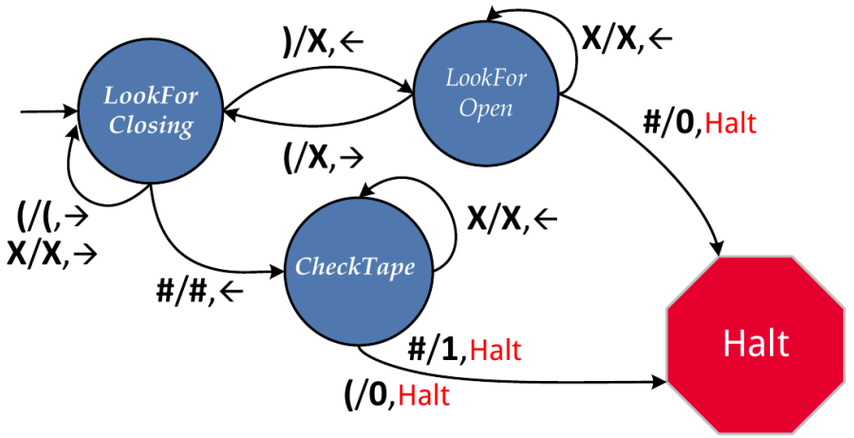
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#



- When `)` is under the head:
- change to `LookForOpen`
- write `X`
- move head to the left
- TM keeps on moving to the left while it sees `X` under its head

Turing Machine tape trace example

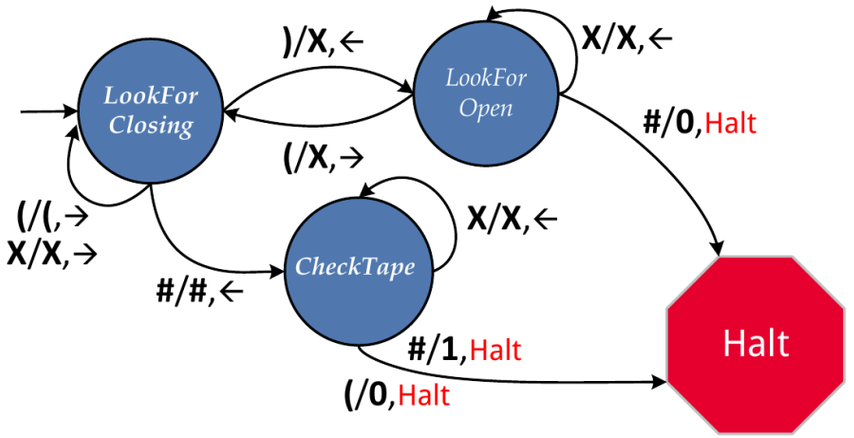
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#
#	(	X	X	(	)	)	#



- When `(` is under the head:
- change to `LookForClosing`
- write `X`
- move head to the right
- TM keeps on moving to the right while it sees `X` or `)` under its head

Turing Machine tape trace example

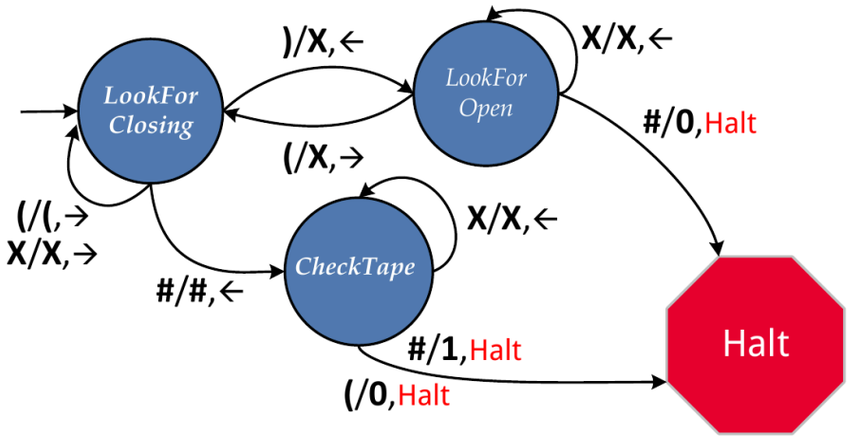
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#
#	(	X	X	(	)	)	#
#	(	X	X	(	X	)	#



- The process continues matching parentheses and switching between states `LookForClosing` and `LookForOpen`

Turing Machine tape trace example

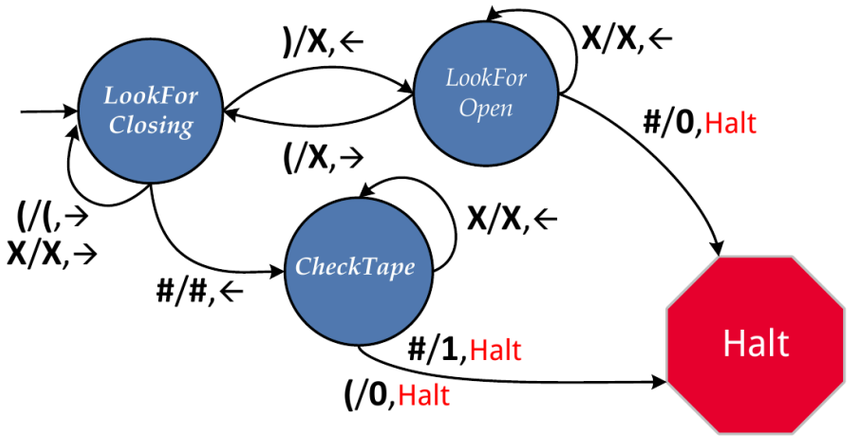
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#
#	(	X	X	(	)	)	#
#	(	X	X	(	X	)	#
#	(	X	X	X	X	)	#
#	(	X	X	X	X	X	#



- The process continues matching parentheses and switching between states `LookForClosing` and `LookForOpen`

Turing Machine tape trace example

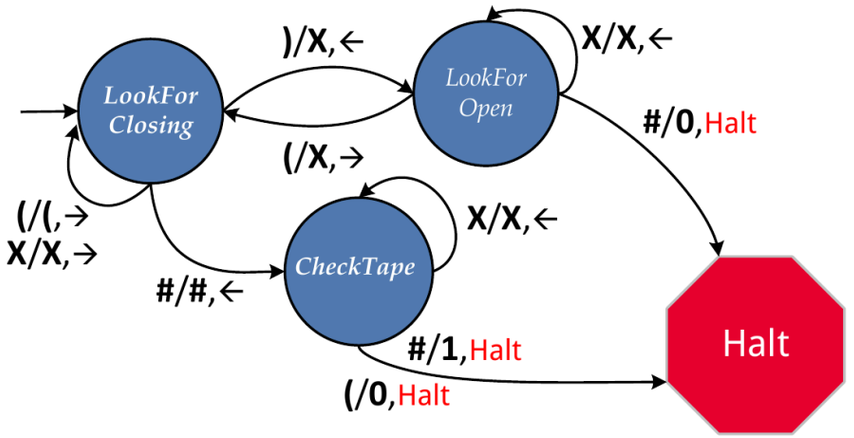
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#
#	(	X	X	(	)	)	#
#	(	X	X	(	X	)	#
#	(	X	X	X	X	)	#
#	(	X	X	X	X	X	#
#	X	X	X	X	X	X	#



- After the last `(` is rewritten, TM is in `LookForClosing` state
- TM goes to the right until it finds `#`
- TM changes to `CheckTape` state
- head starts moving to the left while it sees `X`

Turing Machine tape trace example

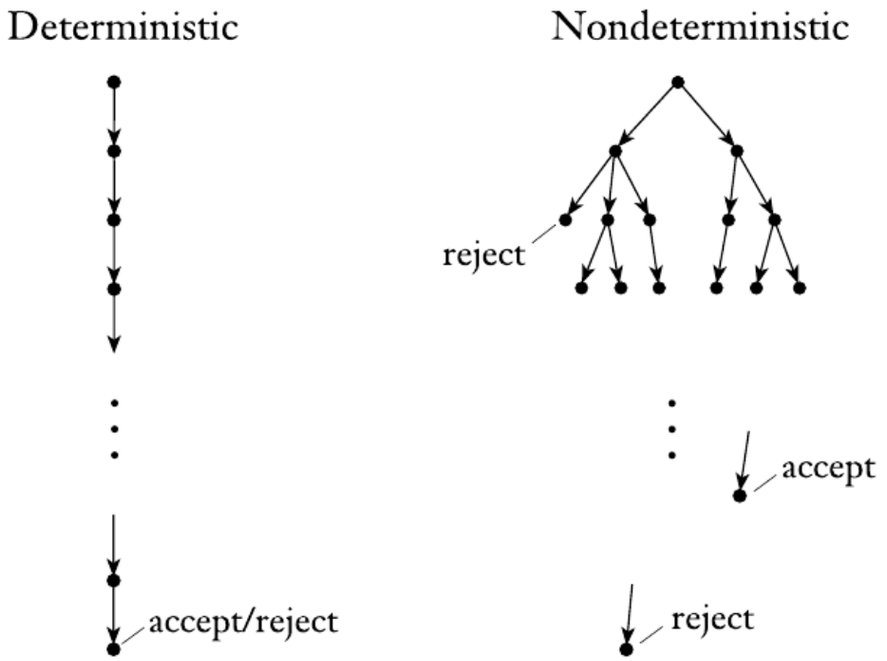
#	(	(	)	(	)	)	#
#	(	(	X	(	)	)	#
#	(	X	X	(	)	)	#
#	(	X	X	(	X	)	#
#	(	X	X	X	X	)	#
#	(	X	X	X	X	X	#
#	X	X	X	X	X	X	#
1	X	X	X	X	X	X	#



- TM finds `#` while in `CheckTape` state
- overwrites it with a `1` (accept)
- TM halts

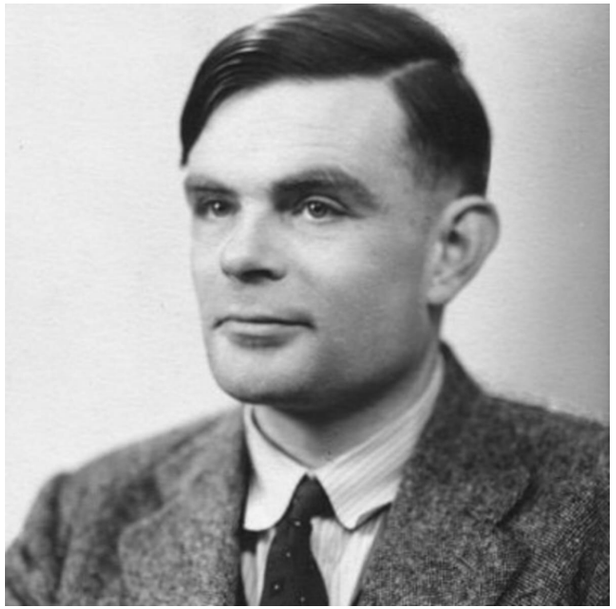
Nondeterministic Turing Machines

- Transition function
  $\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R, N\})$
- A combination of state and tape symbol can go to more than one other state, symbol, and action triplet
- All paths happen **in parallel**
  - alternative: TM takes the “smartest path” towards accept
- Input is **accepted** if any path leads to an accepting state
- Input is **rejected** if all paths lead to a rejecting state or are infinite (loops)



Alan Turing: the father of computation

- Alan Mathison Turing (UK 1912 - 1954)
- Designed the TM to formalize algorithms
- Defined computation as problems that can be solved by algorithms and found examples of problems that cannot be computed
- Famous for his work during WWII to crack the Enigma code with an electromechanical machine called the *Bombe*
- Among the first to work on AI: the Turing test
  - AI as machines able to pass as human a given % of the time when chatting with people
- Died in 1954, by suicide according to the inquest, two years after being prosecuted for his homosexuality and forced to undergo hormone treatment



Computability

Universität Konstanz

1. Finite-state Automata
2. Formal Languages and Grammars
3. Turing Machines
4. **Computability**
5. Complexity Classes

Computability

Universität Konstanz

Definition of algorithm

An algorithm is a self-contained step-by-step set of operations to solve a given problem. Algorithms perform calculation, data processing, and/or automated reasoning tasks.

- Remember: algorithms always **terminate**. For correct inputs, the algorithm must always terminate in a finite number of steps.
- A problem is **computable** if there exists an algorithm that solves the problem.
- If we can prove that no algorithm can solve a problem in finite time, then the problem is **noncomputable**.

The Church-Turing thesis

Universität Konstanz

Church-Turing thesis: intuitive notion of algorithms = Turing Machines

- Up to now, our definition of algorithms is intuitive: we can tell what is an algorithm and what is not, but we lack a mathematical definition
- Alonzo Church and Alan Turing worked in parallel to formally define algorithms and reached equivalent definitions: **an algorithm is what can be computed by a Turing Machine**
- It is a *thesis*: a statement from which the rest of Computer Science is derived but that has no formal proof; its support is that every other model of computation proposed so far is equivalent to a TM or weaker

A noncomputable problem: The halting problem

Universität Konstanz

The Halting Problem

Input: A string representing a program (e.g. in Python)

Output: If evaluating the input program would ever finish, output `True`. Otherwise, output `False`.

- Assume that `halts(X)` is an algorithm that solves the halting problem, where `X` is the Python code that is the input
- `halts(X)` returns `True` if the evaluation of `X` would finish in finite time, `False` otherwise
- For example:
  - `halts('(2 + 3)') = True`
  - `halts('while True: pass') = False`

A noncomputable problem: The halting problem

Universität Konstanz

The Halting Problem

Input: A string representing a program (e.g. in Python)

Output: If evaluating the input program would ever finish, output `True`. Otherwise, output `False`.

- Assume that `halts(X)` is an algorithm that solves the halting problem, where `X` is the Python code that is the input
- `halts(X)` returns `True` if the evaluation of `X` would finish in finite time, `False` otherwise
- For example:
  - `halts('(2 + 3)') = True`
  - `halts('while True: pass') = False`

```
1 halts('n = 4;
2         while sumOfTwoPrimes(n):
3             n = n + 2') = ?
```

Proof of non-computability

- **Proof by reduction to absurdity:**
  - if we assume that `halts(X)` exists, do we reach a contradiction?

```
1 def paradox():
2     if halts('paradox()'):
3         while True:
4             pass
```

- If `halts(X)` exists, then the following is a recursive algorithm:

- if `halts('paradox()') = True`, the algorithm goes into an infinite loop, so `halts('paradox()')` cannot be `True`
- if `halts('paradox()') = False`, the algorithm skips the loop and terminates, so `halts('paradox()')` cannot be `False`
- Since it has to be either `True` or `False`, **`halts(X)` cannot exist**

Proof of non-computability

Universität Konstanz

- **Proof by reduction to absurdity:**
  - if we assume that `halts(X)` exists, do we reach a contradiction?
- If `halts(X)` exists, then the following is a recursive algorithm:

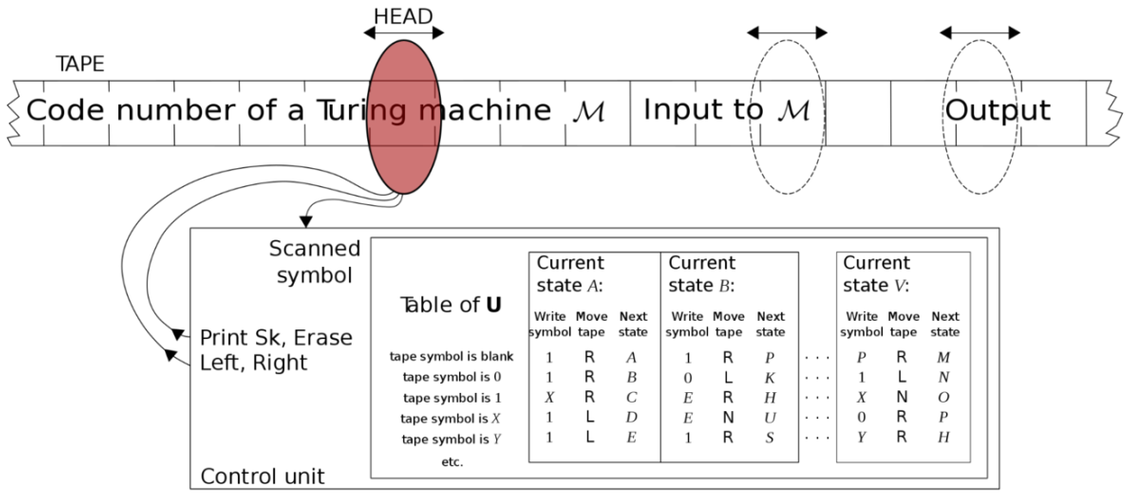
```
1 def paradox():
2     if halts('paradox()'):
3         while True:
4             pass
```

- if `halts('paradox()') = True`, the algorithm goes into an infinite loop, so `halts('paradox()')` cannot be `True`
- if `halts('paradox()') = False`, the algorithm skips the loop and terminates, so `halts('paradox()')` cannot be `False`
- Since it has to be either `True` or `False`, **`halts(X)` cannot exist**

Your knowledge of Python is a model of computing! Alan Turing showed this for Turing Machines as a general model of computing

Universal Turing Machines

- TMs can be encoded with finite codes: e.g. state list + transition table + alphabet, etc.
- A **universal Turing Machine** simulates a TM with a code written on the tape, followed by its input
- The universal TM then writes the output of the simulated TM and stops if the simulated TM stops



https://en.wikipedia.org/wiki/Universal_Turing_machine

Turing completeness

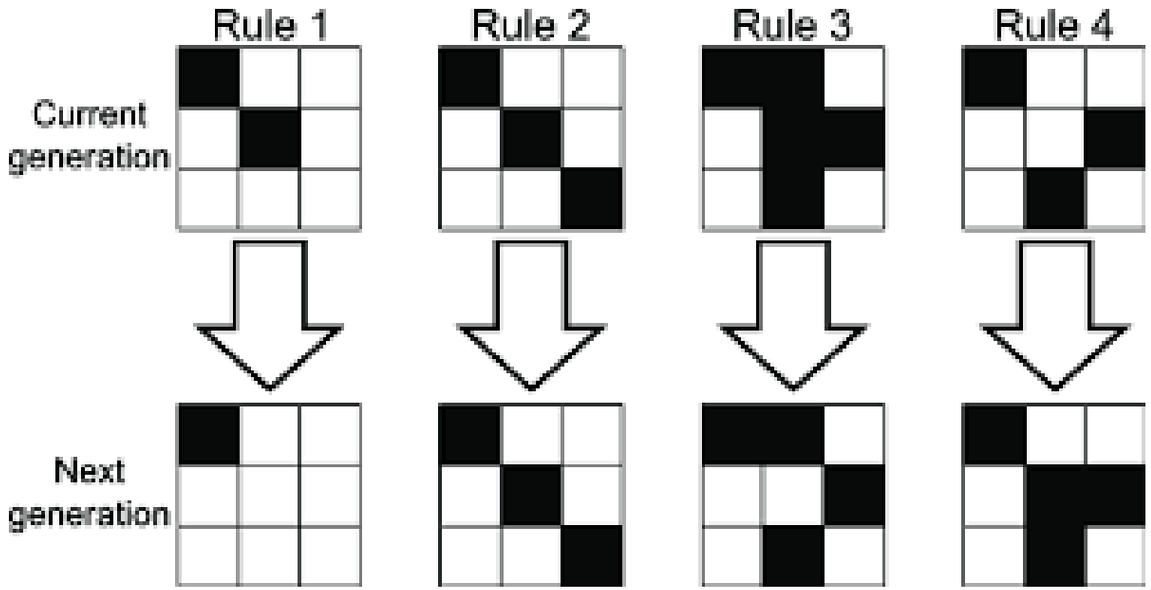
Turing completeness

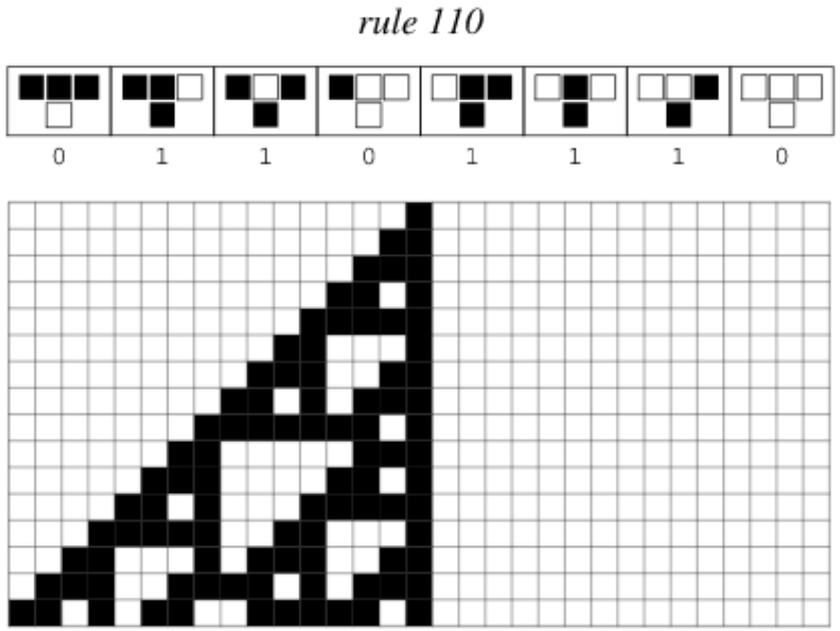
A model of computation (e.g. model of a machine or programming language) is **Turing complete** if it can be used to simulate a universal Turing Machine. Turing complete models can execute any algorithm.

- For example, Python is Turing complete because you can write code in Python that simulates any Turing Machine and thus, by definition, can execute any algorithm
- In practice you can code any algorithm directly in Python without simulating a TM; the point is that you *can* simulate a TM, not that it is easy, understandable, or even efficient

Examples of Turing complete models

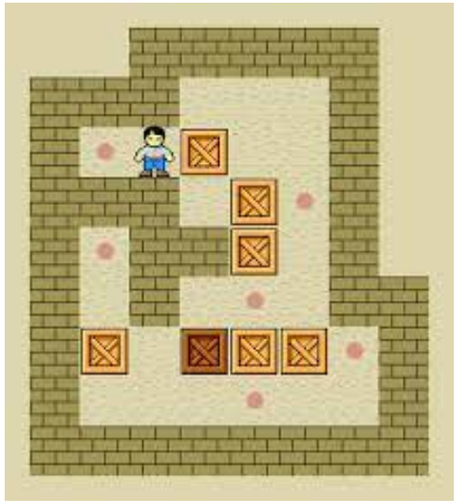
- **Cellular automata:** models of computation in which cells change state according to extremely simple rules
  - Structures emerge that, when interacting, can be set up to simulate any TM
  - Examples are Conway's *Game of Life* and *rule 110*



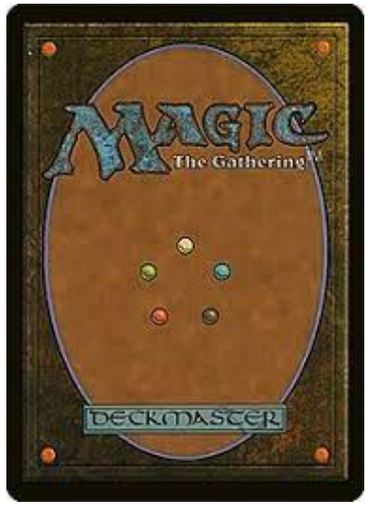


Examples of Turing complete models

- Creative games can be used to build a system that simulates a Universal Turing Machine or other Turing complete models
  - Sokoban (Culberson, 1997: PSPACE-complete, i.e. it simulates any TM with a bounded tape)
  - Minecraft (YouTube video)
  - Magic the Gathering (Churchill, Biderman, & Herrick, 2019)



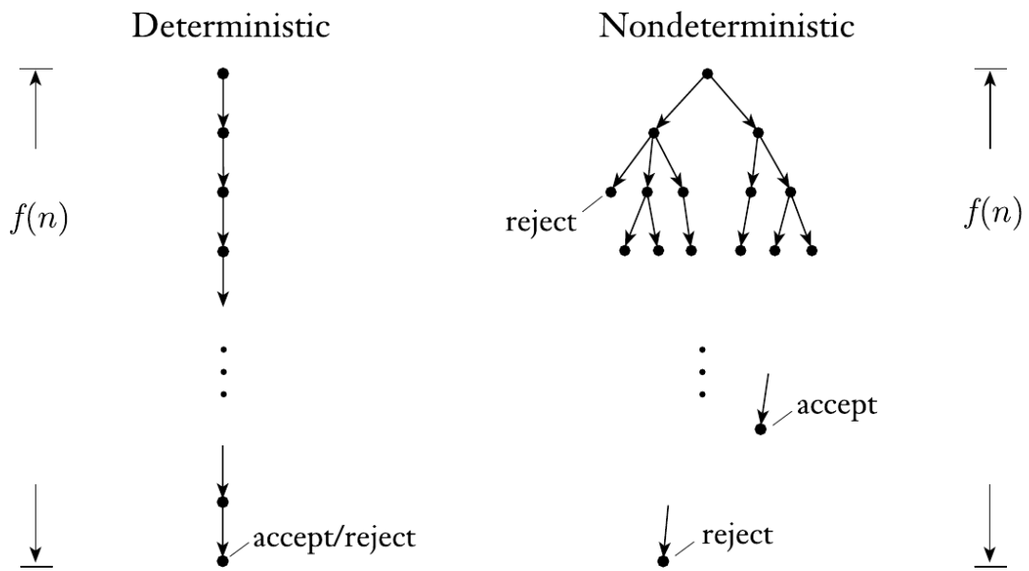




Complexity Classes

1. Finite-state Automata
2. Formal Languages and Grammars
3. Turing Machines
4. Computability
5. **Complexity Classes**

Time complexity of nondeterministic TM



O-notation to characterize scaling of time complexity:

- $O(1) \subset O(\log n) \subset O(n)$
- $O(n) \subset O(n \log n) \subset O(n^2)$
- $O(n^2) \subset O(2^n) \subset O(n!)$
- i.e., ignore lower order terms

- **Recap:** previously, we talked about the time complexity of algorithms: how running time grows with input size
- In terms of a TM, time complexity can be seen as the number of transitions the TM has to do to reach a final state. For a nondeterministic TM, it is the max depth among all execution branches

Problem complexity and decision problems

Universität Konstanz

- We can talk about the complexity of a **problem** regarding the time complexity of known algorithms to solve it
- This defines a hierarchy: inefficient algorithms with long running time can be implemented for almost any problem, but some problems might not be solvable with faster algorithms
- We can define sets of problems as **complexity classes** that relate the problems to the time complexity of the algorithms that can solve them

Decision problem

Decision problems have a binary output (true/false) as the solution. Decision problems can be very general, for example determining if a word belongs to a language is a decision problem.

The P class

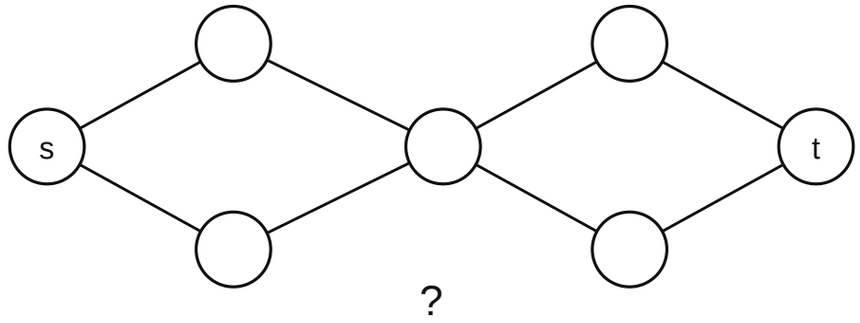
Universität Konstanz

Problem class P

P contains all decision problems that can be solved by a **deterministic** Turing Machine using a **polynomial** amount of computation steps (time).

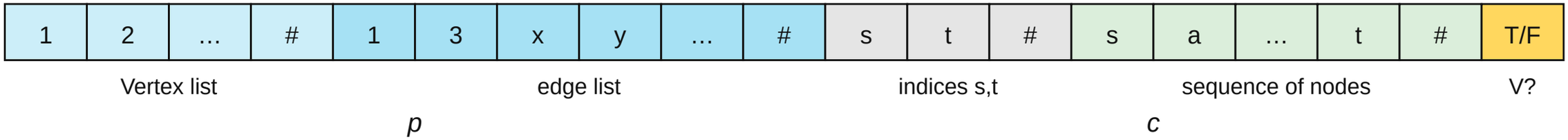
- In other words: for each problem in P, there is at least one algorithm that decides its output in a number of steps that grows polynomially with the input size, e.g. $O(n)$, $O(n^2)$, $O(n^{99})$, also subpolynomial $O(\log n) \subset O(n)$

- An example of a P problem is deciding if there is a path between two vertices in a graph
- You can run DFS on the graph to find the path and since DFS running time is $O(|V| + |E|)$, then the problem belongs to the P class



Problem verifiers

- After solving a problem, you can **verify** if the solution is correct
- A **verifier** is an algorithm (i.e. TM) that takes two inputs:
  - A description (encoding) of a problem p
  - A candidate solution for the problem c
- Verifiers return `True` if c is a solution of p and `False` otherwise
- c is also called *certificate* or *proof* that p has a solution (decision problem)
- However, verifying a solution is always a decision problem
- Example for finding a path between s and t in G:



The NP class

Universität Konstanz

Problem class NP

NP is the set of decision problems for which positive problem instances (the answer is `True`) have proofs **verifiable in polynomial time** by a deterministic Turing Machine.

- An equivalent definition of NP is the set of decision problems solvable in polynomial time by a **nondeterministic** Turing machine
- The nondeterministic TM “guesses” the proof branching and then verifies it deterministically
- NP stands for “**N**ondeterministic **P**olynomial time”
  - It does **not** stand for “Not Polynomial!”

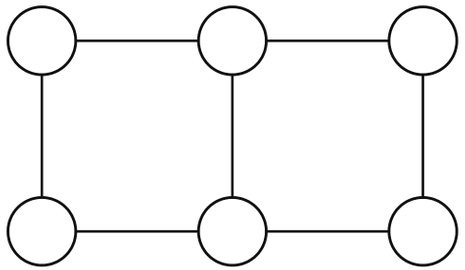
NP problem example: Hamiltonian path

Universität Konstanz

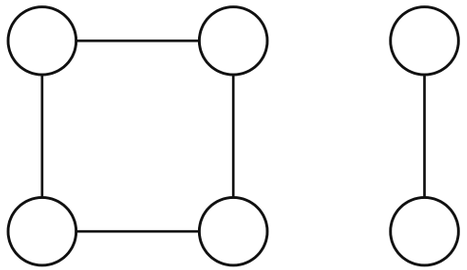
Hamiltonian path

A Hamiltonian path is a path in a graph that visits each vertex exactly once.

G_1 has a Hamiltonian path



G_2 does not have a Hamiltonian path



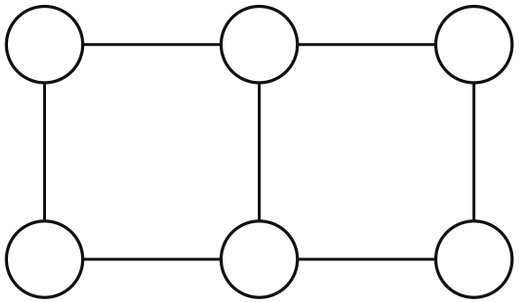
- In this definition of path, edges in the path sequence have to be distinct
- Decision problem: we have to decide if G has a Hamiltonian path or not
  - also known as “is G traceable?”
- Verifying that a path is Hamiltonian in G can be done in $O(|V| + |E|)$ (polynomial)
  - Thus, the problem of deciding if a graph has a Hamiltonian path is NP

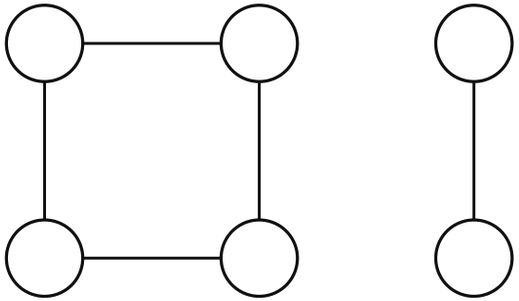
NP problem example: Hamiltonian path

Universität Konstanz

Hamiltonian path

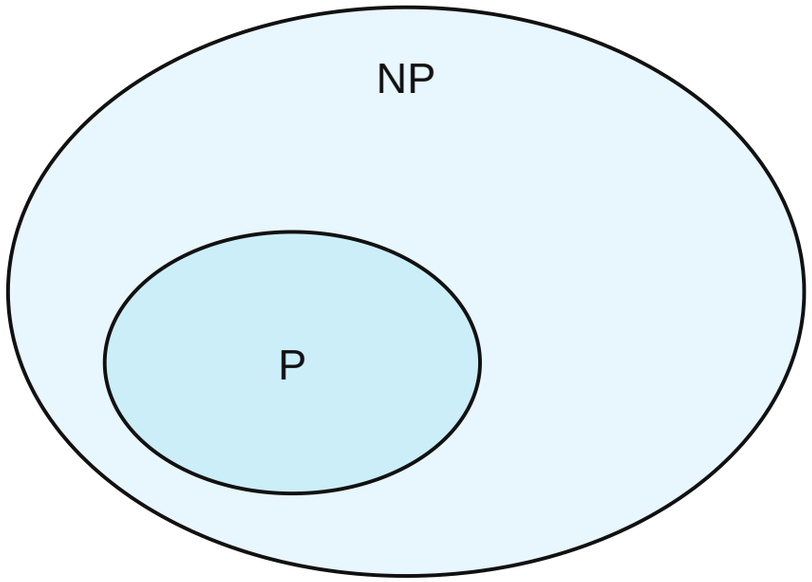
A Hamiltonian path is a path in a graph that visits each vertex exactly once.





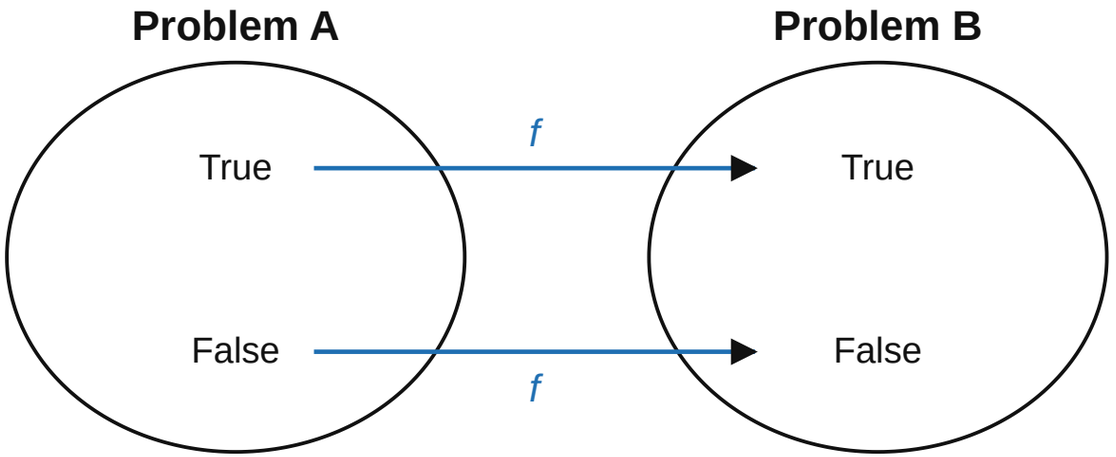
- Finding a Hamiltonian path in G is not that easy!
- No known algorithm can find a Hamiltonian path in polynomial time
- Known algorithms are pretty much like brute force, e.g. trying all combinations: $O(|V|!)$

P vs NP



- All problems in P are also in NP
  - if you can decide it in polynomial time, you can verify it in polynomial time
- Verifying in polynomial time does not imply deciding in polynomial time
- However, this is based on the algorithms known for each problem; an NP problem becomes P when a polynomial-time algorithm for it is invented
- Proving that P = NP or P $\neq$ NP is arguably the most important question in CS
  - A Clay Millennium Problem: 1,000,000 USD prize for whoever solves it

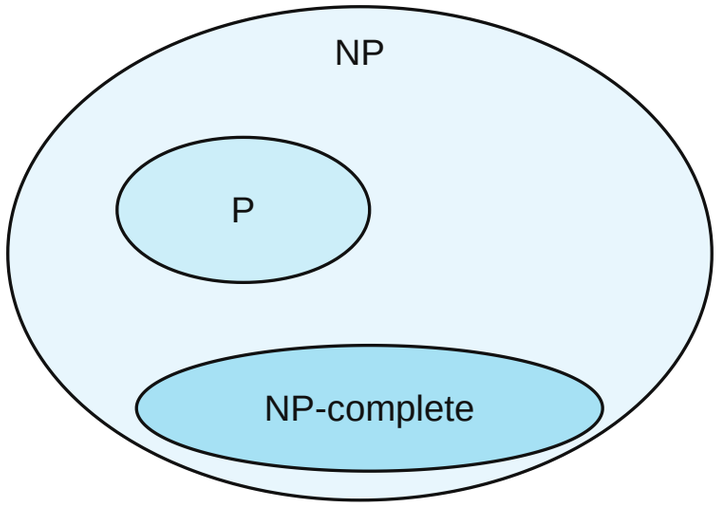
Reducing problems



- **Reducing** a decision problem A to another decision problem B is
  - to find a function f that maps:
  - all inputs of A that have output `True` to all inputs of B that have output `True`
  - all inputs of A that have output `False` to all inputs of B that have output `False`
- An algorithm that solves problem B can be used to solve problem A
- If f can be applied in polynomial time, we say that A is **polynomial time reducible** to B
  - In this case, if B is in class P, then A is in class P too

NP-complete problems

- Some NP problems can be reduced in polynomial time to other NP problems
- This determines a network between problems: if we find a polynomial time algorithm that solves one problem, we solve all problems that can be reduced to it
- Some special NP problems have the property that any other NP problem can be reduced to them:



Problem class NP-complete

A problem is **NP-complete** if:

- it belongs to the NP class
- every other NP problem is polynomial time reducible to it

NP-complete example: boolean satisfiability (SAT)

- Boolean formulas in propositional logic use three operations: AND ($\wedge$), OR ($\vee$), and negation ($\bar{x}$)
- Variables are boolean: they can take value True (1) or False (0)
- The **SAT** problem takes a boolean formula as input and returns True if and only if there is some assignment of variables for which the formula is True
- For example:
  - $(x_1 \vee x_2 \vee x_3 \vee \bar{x}_4) \wedge (\bar{x}_3 \vee x_5 \vee \bar{x}_6) \wedge (x_3 \vee \bar{x}_6)$
    is satisfiable with $x_1 = x_2 = x_4 = x_5 =$ True, $x_3 = x_6 =$ False
  - $(x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_2) \wedge (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$ is not satisfiable
- Any NP problem can be reduced to SAT, making it the original NP-complete problem (proof not covered here)
- **Cook-Levin theorem:** finding a halting execution of any nondeterministic TM can be reformulated as satisfying a logical formula, thus SAT is NP-complete
- If you find a polynomial-time algorithm to solve SAT, then P = NP (and earn $1M)

NP-hard problems

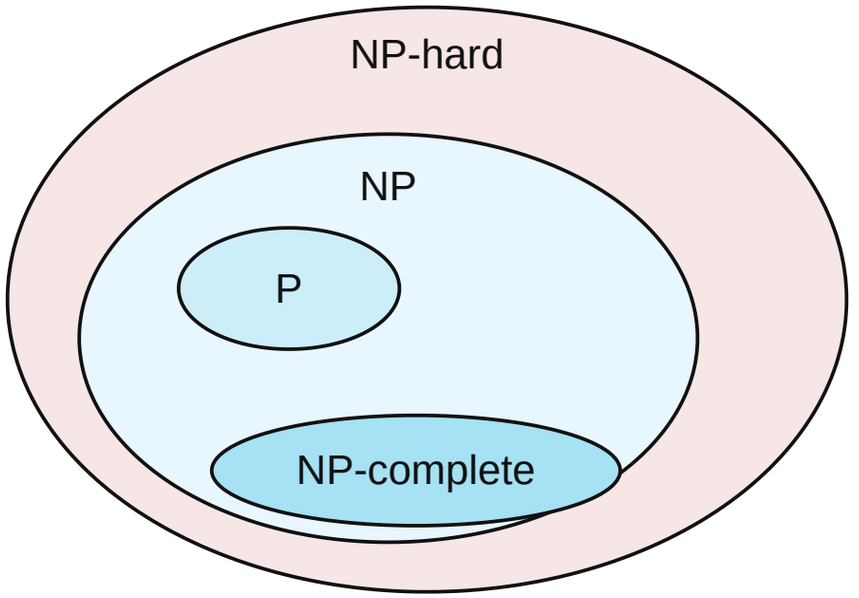
Universität Konstanz

Problem class NP-hard

A problem is **NP-hard** if:

- every NP problem is polynomial time reducible to it

- Note that NP-hard problems do not have to be NP
  - they don't even have to be decision problems
- Informally, they are described as: *at least as hard as the hardest problems in NP*
- Often mentioned to convey the lack of polynomial-time algorithms to solve problems that are not decision problems, for example finding a Hamiltonian path in a graph



Summary: TM, Computability, Complexity Classes

Universität Konstanz

- **Turing Machines**
  - The most general model of a computer we have seen
  - TMs recognize grammars without restrictions
- **Computability**
  - The Church-Turing thesis: Algorithms = TM
  - Some problems cannot be computed: the halting problem
  - A model of computation that can simulate a TM is Turing complete
- **Complexity classes**
  - Problem class P: decision problems solvable in polynomial time
  - Problem class NP: problems with proofs verifiable in polynomial time
  - Polynomial-time reducibility between problems
  - NP-complete problems and their relationship to P vs NP
  - NP-hard: general problems without known efficient algorithms

Next session: Machine Learning